MBLP131 · Hafta 8 · Çalışma Kılavuzu

İşletim Sistemleri Temelleri

Ara Sınav Tekrarı ve Alıştırma: 1–7. Haftalar

Vize için gereken her şey tek yerde: özetler, formüller, çözümlü problemler ve eksiksiz bir deneme sınavı.

Öğretim üyesi: Dr. Öğr. Üyesi Hasan Oğuz
Elektronik Haberleşme Teknolojisi Program Başkanı, Meslek Yüksekokulu
İstanbul Okan Üniversitesi

Bu haftanın sonunda yapabilmeniz gerekenler

1. 1–7. haftaların her temel terimini, somut bir örnekle birlikte bir iki cümleyle açıklamak.
2. Standart diyagramları ezbere çizmek: beş durumlu süreç diyagramı, sistem çağrısının yolu, TLB'li sayfalama donanımı, sayfa hatası adımları, dizin yapıları.
3. Dersteki her sayısal soru türünü çözmek: kullanım oranı ve ek yük formülleri, `fork()` ile süreç sayma, Amdahl yasası, bekleme/dönüş/yanıt süreleriyle Gantt şemaları, ilk uyan, en iyi uyan ve en kötü uyan ile yerleştirme, adres çevirisi, etkin erişim süresi (TLB ve sayfa hataları), sayfa değiştirme çerçeve tabloları ve doğrudan erişim blok hesabı.
4. Dersin uzlaşımlarını (zamanlamada eşitlikler, öncelik sırası, Round Robin'de gelişler, TLB formülü) ifade etmek ve tutarlı biçimde kullanmak.
5. 90 dakikalık eksiksiz bir deneme sınavını yazmak ve çözümlerle kendi kâğıdını puanlamak.

İçindekiler

1 Ara Sınav (Vize) 1

2 1. Hafta: Giriş ve Tarihçe 3

3 2. Hafta: Tanımlar, Hedefler, Bileşenler ve Yapılar 4

4 3. Hafta: İşletim Sistemi Hizmetleri, Sistem Çağrıları ve Süreçler 5

5 4. Hafta: İş Parçacıkları ve İşlemci Zamanlamanın Temelleri 6

6 5. Hafta: Zamanlama Algoritmaları ve Eşzamanlama 7

7 6. Hafta: Bellek Yönetimi 11

8 7. Hafta: Sanal Bellek ve Dosya Sistemleri 13

9 Sık Yapılan Hatalar ve Sınav Stratejisi 15

10 Formül Kâğıdı 16

11 Deneme Sınavı 17

12 Deneme Sınavı: Çözümler 19

13 Sayılar Nasıl Denetlendi? 21

14 İleriye Bakış: 9. Hafta 22

1 Ara Sınav (Vize)

1.1 Pratik bilgiler

Köşeli parantez içindeki maddeler öğretim üyesi tarafından dersin Telegram kanalında kesinleştirilecektir; sınav gününden önce kanala bakın.

Madde	Ayrıntı
Tarih ve saat	[duyurulacak]
Salon	[duyurulacak]
Süre	[90] dakika
Kapsam	1–7. haftalar (dersler, slaytlar ve ders notları)
İzin verilen malzeme	kalem; [basit, programlanamayan bir hesap makinesi — kesinleşecek]; [kitap kapalı, not yok, telefon yok — kesinleşecek]; [formül kâğıdı verilecek mi: evet/hayır — kesinleşecek]
Yanınızda olsun	öğrenci kimliğiniz; 10 dakika erken gelin

1.2 Biçim

Bu haftanın malzemesiyle dağıtılan örnek kâğıt (`exam/MBLP131_Vize_Ornek.pdf`) ve Kısım 11'teki deneme sınavı aynı yapıyı izler; aksi duyurulmadıkça gerçek kâğıt da bu yapıdadır.

Bölüm	Soru türü	Puan	Önerilen süre
A	10 çoktan seçmeli soru, her biri 2 puan	20	15 dk
B	4 kısa yanıtlı soru, her biri 5 puan	20	20 dk
C	5 sayısal problem	60	45 dk
	başta okuma, sonda denetim		$5 + 5$ dk
	Toplam	**100**	**[90] dk**

100 puan için 90 dakikanız olduğuna göre puan başına yaklaşık 0,9 dakikanız var: 2 puanlık bir soru 10 dakika sürmemeli.

1.3 Dersin uzlaşımları: yanıtlarınızda belirtin

Ders kitapları küçük ayrıntılarda farklılaşır; bu yüzden ders aşağıdaki uzlaşımları sabitledi (5. ve 6. haftalar). Sayısal bir yanıtın başına ilgili olanı yazın.

MBLP131'de kullanılan uzlaşımlar

- **Zamanlama:** bağlam değişimi süresi 0 alınır (geçişler ayrıca sayılır); eşitlikler önce varış zamanına, sonra süreç adına göre bozulur; *küçük* öncelik numarası *yüksek* öncelik demektir; Round Robin'de tam bir zaman diliminin bittiği anda gelen süreç hazır kuyruğuna kesilen süreçten *önce* girer.
- **TLB:** arama süresi her erişimde sayılır ve bir ıska fazladan bir bellek erişimine mal olur: EAT $= t_{\text{TLB}} + (2 - \alpha)\, t_m$.
- **Yerleştirme:** delikler adres sırasıyla listelenir; istek seçilen deliğin başından kesilir.
- **Sayfa değiştirme:** bütün çerçeveler boş başlar; yeni sayfa kurbanın çerçevesine girer.

1.4 Bu çalışma kılavuzu nasıl kullanılır?

Kısım 2–8 her hafta için kısa bir özet, formüller, tipik sınav soruları ve bir çözümlü örnek verir; çözümlü örnekler bu haftanın slaytlarında çözülen problemlerdir. Kısım 10 tek sayfalık bir formül kâğıdıdır. Kısım 11 eksiksiz bir deneme sınavıdır: hiçbir şeye bakmadan 90 dakikada yazın, sonra Kısım 12'teki çözümlerle puanlayın. Bu kılavuzdaki her sayı `code/` klasöründeki Python betikleriyle hesaplandı (Kısım 13).

2 1. Hafta: Giriş ve Tarihçe

Özet

Bir bilgisayar komutları izler: girdi $\rightarrow$ işlem $\rightarrow$ çıktı; program bellekte saklanır (**depolanmış program** fikri, stored-program). İlk çalışma tanımı: işletim sistemi, ilk başlayan ve diğer programların çalışabilmesi için her şeyi yöneten programdır; donanımı güvenle paylaştırır ve zor kısımlarını gizler. Tarih, her biri pahalı işlemcinin *boşta* kalması sorununu çözen kuşaklar hâlinde ilerledi:

- **İşletim sistemi yok** (ENIAC, 1945): bir seferde tek iş, programlama kablo panolarını yeniden bağlayarak.

- **Toplu işleme** (batch processing; GM-NAA I/O, 1956, sık sık ilk işletim sistemi sayılır): bir **yerleşik monitör** (resident monitor) işleri art arda yükleyip çalıştırır ve insanın hazırlık süresini ortadan kaldırır.
- **Çoklu programlama** (multiprogramming, 1960'lar): bellekte birkaç iş; biri G/Ç beklerken işlemci başka bir işe geçer.
- **Zaman paylaşımı** (time-sharing; MIT'de CTSS, 1961): her kullanıcı sırayla kısa bir **zaman dilimi** (time quantum) alır ve etkileşimli bir makine deneyimler.
- IBM System/360 ve OS/360 (1964): bir makine ailesi için tek işletim sistemi. Multics (1965–69) **UNIX**'e ilham verdi (Bell Labs, 1969; 1973'te C ile yeniden yazıldı, bu yüzden taşınabilir).
- Kişisel bilgisayarlar (Apple II 1977, IBM PC ve MS-DOS 1981, Macintosh 1984, Windows 95), **Linux** (1991, özgür, UNIX benzeri çekirdek; GNU ile birlikte eksiksiz özgür bir işletim sistemi), akıllı telefonlar (iPhone 2007, Linux çekirdeği üzerinde Android 2008), bulut (AWS 2006, sanal makineler, konteynerler).

1. hafta formülleri

$$U = \frac{t_{\text{çalışma}}}{t_{\text{çalışma}} + t_{\text{hazırlık}}}, \qquad U(n) = 1 - p^{\,n}, \qquad T_{\text{bekleme}} \approx n\,q.$$

U: işlemci kullanım oranı; p: bir işin G/Ç bekleyerek geçirdiği zaman kesri; n: bellekteki iş (ya da kullanıcı) sayısı; q: zaman dilimi.

Tipik sorular

Dönüm noktalarını getirdikleri fikirle eşleştirmek (çoktan seçmeli); çoklu programlama (işlemci verimliliği) ile zaman paylaşımı (kullanıcı deneyimi) arasındaki farkı açıklamak; UNIX'in C ile yeniden yazılmasının neden önemli olduğunu açıklamak; küçük kullanım oranı hesapları.

Slaytlardaki 1(a) problemi: çoklu programlama

İşler zamanlarının $p = 0.6$ kesri kadar G/Ç bekliyor. O zaman $U(1) = 1 - 0.6 = 40\%$, $U(3) = 1 - 0.6^3 = 1 - 0.216 = 78.4\%$ ve $U(6) = 1 - 0.6^6 \approx 95.3\%$. $U \geq 95\%$ sağlayan en küçük n için $0.6^n \leq 0.05$, yani $n \geq \ln 0.05 / \ln 0.6 \approx 5.86$ gerekir; dolayısıyla $n = 6$ (sağlama: $0.6^5 \approx 0.078 > 0.05$, $0.6^6 \approx 0.047 \leq 0.05$).
Toplu işleme: $t_{\text{çalışma}} = 4$ dk ve $t_{\text{hazırlık}} = 6$ dk ise $U = 4/(4+6) = 40\%$; hazırlığı 1 dk'ya indiren bir monitör $U = 4/5 = 80\%$ verir. **Zaman paylaşımı:** $q = 20$ ms ile 30 kullanıcı en çok yaklaşık $n\,q = 30 \times 20$ ms $= 600$ ms bekler.

3 2. Hafta: Tanımlar, Hedefler, Bileşenler ve Yapılar

Özet

- **Çalışma tanımı:** işletim sistemi, donanımı yöneten ve programlara daha basit, daha güvenli bir makine sunan yazılım katmanıdır; bellekte sürekli duran özü **çekirdektir** (kernel). Çekirdek, sistem programları (kabuk, `ls`, derleyici) ve uygulama programları farklı katmanlardır.

- **İki bakış açısı:** yukarıdan aşağı, işletim sistemi temiz soyutlamalar sunan bir *genişletilmiş makinedir* (extended machine; kesimler yerine dosyalar, yazmaçlar yerine süreçler); aşağıdan yukarı, kaynakları zamanda (işlemci, yazıcı) ve uzayda (bellek, disk) çoğullayan bir *kaynak yöneticisidir* (resource manager).
- **Tasarım hedefleri:** kullanım kolaylığı, verimlilik ve gelişebilirlik; birbiriyle çatışırlar ve kişisel bilgisayarlarda, sunucularda, telefonlarda, gömülü ve gerçek zamanlı sistemlerde farklı ağırlıklar alırlar.
- **Dört bileşen:** donanım, işletim sistemi, uygulama programları, kullanıcılar.
- **Kesmeler:** bir aygıt denetleyicisi işlemciye sinyal verir; işlemci durumu kaydeder, **kesme vektöründe** (interrupt vector) bulunan işleyiciyi çalıştırır ve kaldığı yerden sürdürür; işletim sistemi kesme güdümlüdür. Yoklama (polling) işlemci zamanını boşa harcar.
- **Bellek hiyerarşisi:** yazmaçlar (≈ 0.3 ns), önbellek, RAM (≈ 100 ns), SSD ($\approx 50\ \mu$s), HDD (≈ 5 ms); yazmaçlar, önbellek ve RAM uçucudur.
- **İkili mod çalışması** (dual-mode operation): mod biti 0 = çekirdek modu, 1 = kullanıcı modu; kullanıcı modunda ayrıcalıklı bir komut işletim sistemine tuzak (trap) üretir; bir **sistem çağrısı** çekirdek moduna geçer ve geri döner. Ölçtüğümüz bir sistem çağrısı ≈ 116 ns, bir fonksiyon çağrısı ≈ 1.5 ns sürdü.
- **Yapılar:** basit (MS-DOS, koruma yok), monolitik (UNIX, Linux: hızlı, kırılgan), katmanlı (THE: hata ayıklaması kolay, yavaş), mikroçekirdek (MINIX 3, QNX, seL4: güvenilir, daha yavaş), modüler (Linux yüklenebilir çekirdek modülleri), hibrit (Windows, macOS, Android).

2. hafta formülleri

$$\text{yoklama payı} = f \cdot t_{\text{check}}, \qquad T_{\text{mono}} = 2t_s + t_w, \qquad T_{\mu} = 4t_s + 2t_c + t_w.$$

t_s: bir mod geçişi; t_c: bir ileti kopyası; t_w: yararlı iş.

Tipik sorular

İki bakış açısını birer örnekle açıklamak; bir telefon, bir sunucu ve bir fren denetleyicisi için hangi tasarım hedefinin en önemli olduğu; bir kullanıcı programı ayrıcalıklı bir komut yürüttüğünde ne olduğu; `printf()` ile `write()` arasındaki fark; monolitik çekirdeğin ve mikroçekirdeğin birer üstünlüğü ve birer zayıflığı.

Mikroçekirdek maliyeti ve yoklama

$t_s = 0.1\ \mu$s, $t_c = 0.15\ \mu$s ve $t_w = 2\ \mu$s için: $T_{\text{mono}} = 0.2 + 2 = 2.2\ \mu$s, $T_{\mu} = 0.4 + 0.3 + 2 = 2.7\ \mu$s; dolayısıyla mikroçekirdek bu istek için $2.7/2.2 \approx 1.23$ kat yavaştır; yararlı iş uzadıkça oran 1'e yaklaşır. Bir aygıtı saniyede $f = 200$ kez, $t_{\text{check}} = 5\ \mu$s ile yoklamak işlemcinin $200 \times 5 \times 10^{-6} = 10^{-3} = 0.1\%$'ine mal olur.

4 3. Hafta: İşletim Sistemi Hizmetleri, Sistem Çağrıları ve Süreçler

Özet

- Kullanıcı için **hizmetler**: kullanıcı arayüzü, program yürütme, G/Ç işlemleri, dosya sistemi işlemleri, iletişim (ortak bellek ya da ileti aktarımı), hata algılama. Sistemin kendisi için: kaynak ayırma, hesap tutma ve kayıt, koruma ve güvenlik.
- **API, sarmalayıcı, sistem çağrısı:** `printf()` bir API fonksiyonudur; `write()` sarmalayıcısı (wrapper) sistem çağrısı numarasını `rax`'a, parametreleri yazmaçlara koyar ve `syscall` komutunu yürütür; çekirdek numarayı denetler, **sistem çağrısı tablosunda** (system-call table) bulur ve `sys_write()` fonksiyonunu çağırır. `strlen()` hiçbir zaman sistem çağrısı yapmaz.
- **Parametreler** yazmaçlarda, adresi bir yazmaçta duran bir bellek bloğunda ya da yığıtta aktarılır. Altı tür: süreç denetimi, dosya yönetimi, aygıt yönetimi, bilgi bakımı, iletişim, koruma.
- **Süreç** (process), yürütülmekte olan bir programdır; metin, veri, bss, yığın (heap) ve yığıt (stack) bölgeleri vardır. Beş durum: yeni, hazır, çalışıyor, bekliyor, sonlandı; bekliyor durumundan çalışıyor durumuna ok yoktur.
- **PCB** (süreç denetim bloğu) durumu, PID/PPID'yi, program sayacını, yazmaçları, zamanlama, bellek yönetimi ve hesap bilgilerini ve G/Ç durumunu tutar. Bir **bağlam değişimi** (context switch) eski PCB'yi kaydeder ve yenisini yükler; saf ek yüktür.
- `fork()` çocukta 0, ebeveynde çocuğun PID'sini döndürür; `exec()` programı değiştirir (PID aynı kalır); `wait()` çocuğu toplar. Bir **zombi** (zombie) sonlanmış ama henüz beklenmemiştir; bir **yetimin** (orphan) ebeveyni önce sonlanmıştır (Linux onu PID 1'e bağlar).

3. hafta formülleri

$$\text{bağlam değişimi ek yükü} = \frac{s}{q+s};$$

her biri var olan her süreç tarafından yürütülen n `fork()` çağrısı 2^n süreç verir (bunların $2^n - 1$'i yeni oluşturulur).

Tipik sorular

Beş durumlu diyagramı çizip her geçişin ardındaki olayı adlandırmak; PCB alanlarını saymak; sistem çağrılarını altı türe ayırmak; `fork()`'un ne döndürdüğü; zombi ve yetim; ek yük ve `fork()` sayma.

Slaytlardaki 1(b) ve 1(c) problemleri

Ek yük. $s = 4\,\mu$s ile: $q = 2$ ms $= 2000\,\mu$s için ek yük $4/2004 \approx 0.20\%$; $q = 50\,\mu$s için $4/54 \approx 7.4\%$'tür. Çok küçük zaman dilimleri işlemciyi boşa harcar.

fork(). `fork(); fork(); printf("A\n"); fork(); printf("B\n");` Var olan her süreç sonraki her `fork()`'u yürütür. İlk `printf`'ten önce $2 \times 2 = 4$ süreç vardır, dolayısıyla `A` 4 kez yazılır; üçüncü fork onları 8'e katlar ve `B` 8 kez yazılır; toplam 8 süreç, 7'si yeni oluşturulan. (`review_problems.py` içinde `os.fork()` ile yapılan gerçek bir çalıştırma tam olarak 4 `A` ve 8 `B` yazdı. Sınav soruları çıktının terminale gittiğini varsayar; çıktı bir dosyaya yönlendirilirse, 3. haftada gösterildiği gibi, `printf`'in tamponu `fork()` tarafından kopyalanır.)

5 4. Hafta: İş Parçacıkları ve İşlemci Zamanlamanın Temelleri

Özet

- Bir **iş parçacığının** (thread) kendi iş parçacığı kimliği, program sayacı, yazmaçları ve yığıtı vardır; sürecinin kodunu, global verisini, yığınını (heap) ve açık dosyalarını paylaşır. Yararları: tepki verebilirlik, kaynak paylaşımı, ekonomi (makinemizde bir iş parçacığı oluşturmak bir süreç oluşturmaktan yaklaşık 4 kat ucuzdu), ölçeklenebilirlik.
- **Eşzamanlılık** (concurrency): birkaç görev birlikte ilerler; **paralellik** (parallelism): birkaç görev aynı anda çalışır (en az iki çekirdek gerekir).
- Kullanıcı düzeyi iş parçacıkları (hızlı, ama bloklayan bir çağrı bütün süreci bloklar) ve çekirdek düzeyi iş parçacıkları. Modeller: çoktan bire, birebir (Linux, Windows), çoktan çoğa (Go, Java sanal iş parçacıkları).
- Süreçler CPU ve G/Ç patlamaları arasında gidip gelir (işlemci yoğun ve G/Ç yoğun). Kısa vadeli zamanlayıcı hazır kuyruğundan dört karar noktasında seçim yapar (çalışıyor→bekliyor, çalışıyor→hazır, bekliyor→hazır, sonlanma); kesintisiz zamanlama yalnızca ilkini ve sonuncusunu kullanır. **Dağıtıcı** (dispatcher) bağlamı değiştirir, kullanıcı moduna geçer ve kaydedilmiş PC'ye atlar; süresi dağıtım gecikmesidir (dispatch latency).

4. hafta formülleri

$$\text{hızlanma}(N) \le \frac{1}{S + \dfrac{1-S}{N}}, \qquad \lim_{N\to\infty} \text{hızlanma} = \frac{1}{S};$$

$$T_{\text{dönüş}} = T_{\text{tamamlanma}} - T_{\text{geliş}}, \qquad T_{\text{bekleme}} = T_{\text{dönüş}} - T_{\text{patlama}}, \qquad T_{\text{yanıt}} = T_{\text{ilk çalışma}} - T_{\text{geliş}},$$

$$\text{işlemci kullanımı} = \frac{B}{T}, \qquad \text{üretilen iş} = \frac{n}{T}.$$

Kullanımı ve üretilen işi en büyük; dönüş, bekleme ve yanıt süresini en küçük yap. (5. hafta notları aynı büyüklükleri $T_{\text{bitiş}}$ ve $T_{\text{varış}}$ diye yazar: $T_{\text{dönüş}} = T_{\text{bitiş}} - T_{\text{varış}}$.)

Tipik sorular

Hangi öğelerin ortak, hangilerinin özel olduğu (yığıt özeldir, yığın ortaktır); Linux'un hangi modeli kullandığı; eşzamanlılık ve paralellik; Amdahl hızlanmaları; dört karar noktası; bir Gantt şemasından ölçütleri hesaplamak.

Slaytlardaki 1(d) problemi: Amdahl yasası

$S = 0.2$ ile: $N = 2$: $1/(0.2 + 0.4) \approx 1.67$; $N = 4$: $1/(0.2 + 0.2) = 2.5$; $N = 8$: $1/(0.2 + 0.1) \approx 3.33$; sınır $1/0.2 = 5$'tir. Sekiz çekirdek yalnızca 3,33 verir. Tersine, 4 çekirdekte en az 3 kat hızlanma $S + (1-S)/4 \le 1/3$, yani $S \le 1/9 \approx 0.11$ gerektirir.

6 5. Hafta: Zamanlama Algoritmaları ve Eşzamanlama

Özet: zamanlama

Algoritma	Kural	Kesintili	Hatırlayın
FCFS	FIFO hazır kuyruğu; bitene kadar çalışır	hayır	konvoy etkisi
SJF	en kısa sonraki CPU patlaması önce	hayır	hepsi hazırken en iyi ortalama bekleme; patlama tahmin edilmeli
SRTF	*kalan* süreden kısa olan yeni gelen işlemciyi alır	evet	en küçük ortalama bekleme; uzun işler aç kalabilir
Öncelikli	en yüksek öncelik (en küçük sayı) önce	ikisi de	açlık; çaresi: yaşlandırma
Döngüsel (RR)	zaman dilimi q; kesilen süreç kuyruğun sonuna	evet	q çok büyük $\rightarrow$ FCFS; çok küçük $\rightarrow$ ek yük

Linux sıradan görevleri adalet için zamanlar (CFS, 6.6 çekirdeğinden beri EEVDF), nice değeriyle ağırlıklandırarak; çok düzeyli geri beslemeli kuyruklar davranışı izleyerek SJF'ye yaklaşır.

Özet: eşzamanlama

- Birkaç iş parçacığı aynı veriye eşzamanlı eriştiğinde, en az biri yazdığında ve sonuç sıraya bağlı olduğunda bir **yarış durumu** (race condition) oluşur; `counter++` oku–ekle–yaz demektir.
- **Kritik bölge problemi** (critical-section problem): giriş, kritik, çıkış ve kalan bölümler; bir çözüm *karşılıklı dışlama* (mutual exclusion), *ilerleme* (progress) ve *sınırlı bekleme* (bounded waiting) sağlamalıdır.
- Kesmeleri kapatmak çok işlemcide işe yaramaz ve ayrıcalıklıdır; Peterson çözümü (iki iş parçacığı; modern işlemcilerde bellek bariyerleri gerektirir); atomik `test_and_set` ve karşılaştır-ve-değiştir (CAS); **muteks kilitleri** (mutex locks: `acquire`/`release`; döner kilit ya da uyuyarak bekleme).
- Bir **semafor** S'ye yalnızca `wait(S)` (P: `while (S <= 0); S--;`) ve `signal(S)` (V: `S++`) ile erişilir; ikili (başlangıç 1, muteks gibi) ya da sayan (başlangıç N). Sınırlı tampon: `empty` $= n$, `full` $= 0$, `mutex` $= 1$; `wait(empty)`, `wait(mutex)`'ten *önce* gelir.
- **Kilitlenme** (deadlock): bir gruptaki her iş parçacığı yalnızca grubun başka bir üyesinin yol açabileceği bir olayı bekler; kilitleri tek bir genel sırayla alarak önlenir. Açlık farklıdır: başkaları ilerlemeye devam eder.

5. hafta formülleri

$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n,$ RR: sonraki sıra için bekleme $\leq (n-1)\,q,$ geçiş ek yükü $= \dfrac{s}{q+s}.$

Tipik sorular

İki üç algoritma için Gantt şeması ve bitiş, dönüş, bekleme, yanıt süreleriyle ortalamalarını içeren bir tablo; üstel ortalama; `counter++` için kaybettiren bir iç içe geçme; üç gereksinim; semafor izleri; üreticideki iki `wait` çağrısının sırasının neden önemli olduğu.

6.1 Slaytlardaki 2. problem: tek iş kümesi, beş zamanlayıcı

İşler (süreler ms): P1 (varış 0, patlama 7, öncelik 2), P2 (2, 4, 1), P3 (3, 2, 4), P4 (5, 5, 3). Toplam iş 18 ms'dir ve işlemci hiç boşta kalmaz. Üretilmiş tablolar:

FCFS



Süreç	Varış	Patlama	Öncelik	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	2	7	$7-0=7$	$7-7=0$	0
P2	2	4	1	11	$11-2=9$	$9-4=5$	5
P3	3	2	4	13	$13-3=10$	$10-2=8$	8
P4	5	5	3	18	$18-5=13$	$13-5=8$	8
Ortalama					$39/4=9.75$	$21/4=5.25$	$21/4=5.25$

SJF (kesintisiz)



$t=7$'de hazır kuyruğunda P2 (4), P3 (2) ve P4 (5) vardır: en kısası P3'tür.

Süreç	Varış	Patlama	Öncelik	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	2	7	$7-0=7$	$7-7=0$	0
P2	2	4	1	13	$13-2=11$	$11-4=7$	7
P3	3	2	4	9	$9-3=6$	$6-2=4$	4
P4	5	5	3	18	$18-5=13$	$13-5=8$	8
Ortalama					$37/4=9.25$	$19/4=4.75$	$19/4=4.75$

SRTF



$t = 2$: P2'nin ihtiyacı $4 <$ P1'in kalanı 5: kesme. $t = 3$: P3'ün ihtiyacı $2 <$ P2'nin kalanı 3: kesme. $t = 5$: kalanlar P1 5, P2 3, P4 5: P2 çalışır. $t = 8$: P1 ve P4'ün ikisi de 5 ister: eşitlik erken gelen P1'in lehine bozulur.

Süreç	Varış	Patlama	Öncelik	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	2	13	$13 - 0 = 13$	$13 - 7 = 6$	0
P2	2	4	1	8	$8 - 2 = 6$	$6 - 4 = 2$	0
P3	3	2	4	5	$5 - 3 = 2$	$2 - 2 = 0$	0
P4	5	5	3	18	$18 - 5 = 13$	$13 - 5 = 8$	8
Ortalama					$34/4 = 8.5$	$16/4 = 4$	$8/4 = 2$

Öncelikli (kesintili)



P2 (öncelik 1) $t = 2$'de işlemciyi P1'den (öncelik 2) alır; P3'ün önceliği en düşüktür ve en son çalışır.

Süreç	Varış	Patlama	Öncelik	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	2	11	$11 - 0 = 11$	$11 - 7 = 4$	0
P2	2	4	1	6	$6 - 2 = 4$	$4 - 4 = 0$	0
P3	3	2	4	18	$18 - 3 = 15$	$15 - 2 = 13$	13
P4	5	5	3	16	$16 - 5 = 11$	$11 - 5 = 6$	6
Ortalama					$41/4 = 10.25$	$23/4 = 5.75$	$19/4 = 4.75$

Round Robin, $q = 3$



$t = 3$: P3 tam P1'in zaman dilimi biterken gelir; dolayısıyla kuyruk P2, P3, P1'dir. $t = 6$: P4 (5'te geldi) P1'in arkasındadır ve P2 (1 ms kaldı) sona katılır: P3, P1, P4, P2. $t = 8$: P3 biter; P1 8–11 arası çalışır ve 1 ms'si kalır; sonra P4 11–14, P2 14–15 (biter), P1 15–16 (biter), P4 16–18.

Süreç	Varış	Patlama	Öncelik	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	7	2	16	$16 - 0 = 16$	$16 - 7 = 9$	0
P2	2	4	1	15	$15 - 2 = 13$	$13 - 4 = 9$	1
P3	3	2	4	8	$8 - 3 = 5$	$5 - 2 = 3$	3
P4	5	5	3	18	$18 - 5 = 13$	$13 - 5 = 8$	6
Ortalama					$47/4 = 11.75$	$29/4 = 7.25$	$10/4 = 2.5$

Algoritma	Kesintili	Ort. bekleme	Ort. dönüş	Ort. yanıt	Geçiş
FCFS	hayır	5.25	9.75	5.25	3
SJF	hayır	4.75	9.25	4.75	3
SRTF	evet	4	8.5	2	5
Öncelikli	evet	5.75	10.25	4.75	4
RR, $q = 3$	evet	7.25	11.75	2.5	7

SRTF, kuramın söylediği gibi en küçük ortalama bekleme süresini verir; öncelikli zamanlama öneme hizmet

eder (P3 13 ms bekler); RR'nin ortalama beklemesi en büyüktür ama ortalama yanıt süresi iyidir.

Üstel ortalama

$\alpha = 0.5$, $\tau_0 = 8$ ms, ölçülen patlamalar $t_0, t_1, t_2 = 6, 10, 4$ ms: $\tau_1 = 0.5 \cdot 6 + 0.5 \cdot 8 = 7$, $\tau_2 = 0.5 \cdot 10 + 0.5 \cdot 7 = 8.5$, $\tau_3 = 0.5 \cdot 4 + 0.5 \cdot 8.5 = 6.25$ ms.

6.2 Slaytlardaki 3. problem: bir yarış ve bir semafor

`counter` $= 10$; A iş parçacığı `counter++`, B iş parçacığı `counter--` yürütür; her biri kendi yazmacına oku–ekle–yaz yapar. Talihsiz bir sıra:

Adım	A iş parçacığı	B iş parçacığı	counter
1	oku (yazmaç $= 10$)		10
2		oku (yazmaç $= 10$)	10
3	1 ekle (yazmaç $= 11$)		10
4		-1 ekle (yazmaç $= 9$)	10
5	yaz		11
6		yaz	9 (10 olmalı)

İç içe geçmeler (her iş parçacığı kendi sırasını korur) $\binom{6}{3} = 20$ tanedir. `review_problems.py` betiği hepsini sayar: son değer 9 sırada 9, 2 sırada 10 ve 9 sırada 11'dir: 20 sıranın yalnızca 2'si doğrudur. Çözüm, bir muteksle korunan kritik bölgedir.

Sayan semafor $S = 2$ (iki yazıcı), T1, T2, T3 iş parçacıkları: T1 `wait` $\rightarrow$ girer, $S = 1$; T2 `wait` $\rightarrow$ girer, $S = 0$; T3 `wait` $\rightarrow$ bloklanır; T1 `signal` $\rightarrow S = 1$ ve T3 hemen `wait`'ini geçer, $S = 0$; T2 `signal` $\rightarrow S = 1$; T3 `signal` $\rightarrow S = 2$. Herhangi bir anda en çok iki iş parçacığı yazıcı kullanır.

7 6. Hafta: Bellek Yönetimi

Özet

- **Adres bağlama** (address binding) derleme zamanında (mutlak kod), yükleme zamanında (yeri değiştirilebilir kod) ya da yürütme zamanında (**MMU** gerekir; bütün modern sistemler) yapılır. Mantıksal (sanal) adresleri işlemci üretir; fiziksel adresleri bellek görür.
- **Taban ve sınır:** MMU $0 \leq a <$ sınır koşulunu denetler ve tabanı ekler; değilse tuzak üretir. Bu yazmaçları yüklemek ayrıcalıklıdır.
- Bütün süreçleri **takas** (swapping) etmek saniyeler sürer; modern sistemler sayfaları takas eder.
- Değişken bölümlerle **bitişik yer ayırma**: ilk uyan, en iyi uyan, en kötü uyan. **Dış parçalanma**: yeterince boş bellek var ama bitişik değil (yüzde 50 kuralı: üçte bire kadar kayıp); **iç parçalanma**: ayrılmış bir bloğun içindeki kullanılmayan bellek. Sıkıştırma (compaction) yürütme zamanında bağlama gerektirir.
- **Sayfalama:** eşit boyda (2^n) çerçeveler (fiziksel) ve sayfalar (mantıksal); sayfa tablosu $p \rightarrow f$

eşlemesini tutar; dış parçalanma yok, süreç başına ortalama yarım sayfa iç parçalanma; geçerli–geçersiz ve koruma bitleri; paylaşılan (yeniden girilebilir) kod sayfaları; çok düzeyli sayfa tabloları.

- **TLB:** (p, f) çiftlerinin küçük, ilişkili (associative) bir önbelleği; yerellik sayesinde isabet oranı yüksektir.
- **Bölütleme** (segmentation): adres $\langle s, d \rangle$, (taban, sınır) çiftlerinden oluşan bölüt tablosu; değişken boylar dış parçalanmayı geri getirir. x86-64 Linux düz bölütler kullanır ve bütün çeviriyi sayfalamayla yapar.

6. hafta formülleri

$$0 \le a_{\text{mant}} < \text{sınır} \;\Rightarrow\; a_{\text{fiz}} = \text{taban} + a_{\text{mant}}, \text{ değilse tuzak;} \qquad t_{\text{takas}} = \frac{\text{boyut}}{\text{aktarım hızı}};$$

$$p = \left\lfloor \frac{a}{2^n} \right\rfloor, \quad d = a \bmod 2^n, \quad a_{\text{fiz}} = f \cdot 2^n + d; \qquad \text{girdi sayısı} = \frac{2^m}{2^n};$$

$$\text{EAT} = \alpha\,(t_{\text{TLB}} + t_m) + (1-\alpha)\,(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m;$$

$$\text{sadeleştirilmiş } (t_{\text{TLB}} \approx 0)\text{: EAT} = \alpha\, t_m + (1-\alpha)\, 2t_m;$$

$$0 \le d < \text{sınır}[s] \;\Rightarrow\; a_{\text{fiz}} = \text{taban}[s] + d, \text{ değilse tuzak.}$$

α: TLB isabet oranı (hit ratio); t_m: bir bellek erişiminin süresi.

Tipik sorular

Üç bağlama zamanı; mantıksal ve fiziksel adresleri kimin gördüğü; onluk ve onaltılık tabanda taban/sınır çevirisi; takas süreleri; üç uyan stratejisiyle yerleştirme; iç ve dış parçalanma; bir sayfa tablosu üzerinden adres çevirisi; sayfa tablosu boyutu; TLB ile EAT ve hedef bir EAT için gereken isabet oranı; bölüt tablosuyla çeviri; sayfalama ve bölütleme karşılaştırması.

Taban ve sınır, takas

Taban 12000, sınır 2500: $a = 100 \to 12100$; $a = 2499 \to 14499$ (son geçerli bayt); $a = 2500$: **tuzak** (denetim katıdır). 200 MB'lık bir süreci 80 MB/s hızla takas etmek her yön için $200/80 = 2.5$ s, dışarı artı içeri 5 s sürer.

7.1 Slaytlardaki 4. problem: ilk uyan, en iyi uyan, en kötü uyan

Delikler (boyut@başlangıç, KB, adres sırasında): $300@100$, $600@500$, $200@1300$, $450@1700$, $350@2400$ (toplam 1900 KB). Sırayla istekler: A $= 340$, B $= 180$, C $= 420$, D $= 500$ KB. Üretilmiş tablo:

İstek	İlk uyan	En iyi uyan	En kötü uyan
A $= 340$	$600@500 \to$ kalan 260	$350@2400 \to$ kalan 10	$600@500 \to$ kalan 260
B $= 180$	$300@100 \to$ kalan 120	$200@1300 \to$ kalan 20	$450@1700 \to$ kalan 270
C $= 420$	$450@1700 \to$ kalan 30	$450@1700 \to$ kalan 30	beklemek zorunda
D $= 500$	beklemek zorunda	$600@500 \to$ kalan 100	beklemek zorunda

İlk uyan: 960 KB boş olduğu hâlde D beklemek zorundadır, çünkü en büyük delik yalnızca 350 KB'tır (dış parçalanma). En iyi uyan dördünü de yerleştirir. En kötü uyan iki kez başarısız olur (1380 KB boş, en büyük

delik 350 KB). Tek bir örnek genel bir şey kanıtlamaz: örnek kâğıtta ilk uyan kazanır, en iyi uyan başarısız olur.

7.2 Slaytlardaki 5. problem: sayfalama ve TLB

16 bitlik adresler, 4 KB'lık sayfalar ($n = 12$): p ilk onaltılık basamak, d son üç basamaktır. Sayfa tablosu: $0 \to 3$, $1 \to 8$, $2 \to 6$, $4 \to 10$, $5 \to 2$; 3. sayfa ve 6–15 arası sayfalar geçersiz.

- `0x1C3B` $= 7227$: $p = 1$, $d =$ `0xC3B` $= 3131$; 8. çerçeve: $8 \times 4096 + 3131 = 35899 =$ `0x8C3B`.
- `0x4F0E` $= 20238$: $p = 4$, $d =$ `0xF0E` $= 3854$; $10 =$ `0xA`. çerçeve: `0xAF0E` $= 44814$.
- `0x2010`: $p = 2$, 6. çerçeve: `0x6010` $= 24592$.
- `0x3004`: $p = 3$ geçersizdir: **tuzak**, fiziksel adres yok.

9000 B'lık bir süreç $\lceil 9000/4096 \rceil = 3$ çerçeveye ihtiyaç duyar ($9000 = 2 \times 4096 + 808$); dolayısıyla $4096 - 808 = 3288$ B iç parçalanmaya gider. 32 bitlik adresler, 8 KB'lık sayfalar ve 4 baytlık girdilerle düz bir sayfa tablosunda $2^{32}/2^{13} = 2^{19} = 524\,288$ girdi, yani 2^{21} B $= 2$ MB vardır.

TLB: $t_{\text{TLB}} = 10$ ns, $t_m = 80$ ns, $\alpha = 0.95$:

$$\text{EAT} = 0.95 \times 90 + 0.05 \times 170 = 85.5 + 8.5 = 94 \text{ ns} = 10 + (2 - 0.95) \times 80.$$

TLB olmadan her erişim $2t_m = 160$ ns sürer; sadeleştirilmiş formül $0.95 \times 80 + 0.05 \times 160 = 84$ ns verir. EAT $= 100$ ns için: $10 + (2 - \alpha)\,80 = 100 \Rightarrow 2 - \alpha = 1.125 \Rightarrow \alpha = 0.875$.

Bölüt tablosu

0. bölüt: taban 1000, sınır 800; 1. bölüt: taban 5200, sınır 300; 2. bölüt: taban 2400, sınır 1500. $\langle 2, 1499 \rangle$: $1499 < 1500 \Rightarrow 2400 + 1499 = 3899$. $\langle 1, 300 \rangle$: $300 \geq 300 \Rightarrow$ **tuzak**. $\langle 0, 799 \rangle$: $799 < 800 \Rightarrow 1000 + 799 = 1799$.

8 7. Hafta: Sanal Bellek ve Dosya Sistemleri

Özet: sanal bellek

- **Sanal bellek** (virtual memory) RAM'de yalnızca kullanılan sayfaları tutar; geri kalanlar yedek depoda (backing store) durur. **Başvuru yerelliği** (locality of reference) sayesinde çalışır.
- **İsteğe bağlı sayfalama** (demand paging) bir sayfayı ilk kullanıldığında yükler; geçerli–geçersiz biti sayfanın bellekte olup olmadığını söyler. Bir **sayfa hatası** (page fault) altı adımda işlenir: başvuru, tuzak, bulma (diskteki sayfa ve boş bir çerçeve), okuma, sayfa tablosunu güncelleme, komutu yeniden başlatma. Küçük hatalar (minor) disk G/Ç'si gerektirmez, büyük hatalar (major) gerektirir.
- **Yazınca kopyala** (copy-on-write): `fork()`'tan sonra sayfalar, bir süreç yazana kadar salt okunur olarak paylaşılır; yalnızca yazılan sayfa kopyalanır.
- **Sayfa değiştirme**: FIFO (en önce yüklenen; Belady anomalisi olabilir), OPT (en uzak gelecekte kullanılacak; en az hata; geleceği ister), LRU (en uzun süredir kullanılmayan; başvuru bitiyle saat / ikinci şans algoritmasıyla yaklaşılır). Değişiklik (kirli) biti, temiz kurbanların diske geri yazılmasını önler.

- **Boşa çalışma** (thrashing): süreçler yürütmekten çok sayfalamaya zaman harcar; çaresi çoklu programlama derecesini düşürmektir. **Çalışma kümesi** (working set) $\mathrm{WS}(t, \Delta)$, son Δ başvurudaki farklı sayfaların kümesidir.

Özet: dosya sistemleri

- Bir **dosyanın** öznitelikleri (ad, kimlik, tür, konum, boyut, koruma, zaman damgaları) dosya denetim bloğunda tutulur (Linux'ta **düğüm**, inode; ad dizindedir). İşlemler: create, open, read, write, seek, close, delete (`unlink`), truncate. `open()` bir dosya tanımlayıcısı (file descriptor) döndürür; süreç başına tablolar dosya işaretçisini, sistem geneli tablo açılma sayısını tutar.
- **Erişim yöntemleri**: sıralı (teyp modeli), doğrudan (herhangi bir n. blok; disk modeli), indeksli (küçük bir indekste ara, sonra tek bir blok oku).
- **Dizinler**: tek düzeyli, iki düzeyli (UFD/MFD), ağaç (mutlak ve göreli yollar, `.` ve `..`), döngüsüz çizge (bağlantılar), genel çizge (döngüler; uygulamada kaçınılır).
- Bir **sabit bağlantı** (hard link), aynı düğüm için başka bir dizin girdisidir (bağlantı sayısı, link count); bir **sembolik bağlantı** (symbolic link) bir yol içeren küçük bir dosyadır ve hedef silinirse boşta kalır (dangling).

7. hafta formülleri

$$\mathrm{EAT} = (1-p)\,t_m + p\,t_{\text{fault}};$$

$$\text{bayt konumu} = k\,R, \quad \text{blok} = \left\lfloor \frac{kR}{B} \right\rfloor, \quad \text{blok içi konum} = kR - \text{blok} \cdot B;$$

$$D = \sum_i \mathrm{WSS}_i > m \;\Rightarrow\; \text{boşa çalışma.}$$

p: sayfa hatası oranı; t_{fault}: sayfa hatası hizmet süresi; k: kayıt numarası (0'dan başlar); R: kayıt boyu; B: blok boyu; m: kullanılabilir çerçeve sayısı. Kaydın *son* baytını, $kR+R-1$'i de her zaman denetleyin.

Tipik sorular

Sayfa hatasının altı adımı; komutun neden yeniden başlatıldığı; yazınca kopyala ve `fork()`; FIFO, LRU ve OPT için çerçeve tabloları; Belady anomalisi; boşa çalışma ve çalışma kümesi; sayfa hatalarıyla EAT ve belirli bir yavaşlama için en büyük hata oranı; blok hesabı; erişim yöntemlerini, dizin yapılarını, sabit ve sembolik bağlantıları karşılaştırmak.

8.1 Slaytlardaki 6. problem: sayfa değiştirme

3 çerçeve, başlangıçta boş, başvuru dizisi $1, 2, 0, 4, 2, 3, 2, 1, 2, 4, 0, 2, 0, 1$. Nane: yeni yüklenen sayfa; soluk: hata sütunu; gri: isabet. Üretilmiş tablolar:

FIFO: 11 hata

başvuru	1	2	0	4	2	3	2	1	2	4	0	2	0	1
çerçeve 0	1	1	1	4	4	4	4	1	1	1	1	2	2	2
çerçeve 1		2	2	2	2	3	3	3	3	4	4	4	4	1
çerçeve 2			0	0	0	0	2	2	2	2	0	0	0	0
hata?	F	F	F	F		F	F	F		F	F	F		F

LRU: 9 hata

başvuru	1	2	0	4	2	3	2	1	2	4	0	2	0	1
çerçeve 0	1	1	1	4	4	4	4	1	1	1	0	0	0	0
çerçeve 1		2	2	2	2	2	2	2	2	2	2	2	2	2
çerçeve 2			0	0	0	3	3	3	3	4	4	4	4	1
hata?	F	F	F	F		F		F		F	F			F

OPT: 7 hata

başvuru	1	2	0	4	2	3	2	1	2	4	0	2	0	1
çerçeve 0	1	1	1	1	1	1	1	1	1	1	1	1	1	1
çerçeve 1		2	2	2	2	2	2	2	2	2	2	2	2	2
çerçeve 2			0	4	4	3	3	3	3	4	0	0	0	0
hata?	F	F	F	F		F				F	F			

FIFO: bir isabet (5. konumdaki 2 başvurusu) geliş sırasını değiştirmez. LRU, 3'e başvuruda: çerçevelerde $4, 2, 0$ vardır ve en uzun süredir kullanılmayan 0'dır; böylece sık kullanılan 2 kalır. OPT, 4'e başvuruda: sonraki kullanımlar 2 (5. konum), 1 (8. konum), 0 (11. konum); dolayısıyla 0 çıkar. Dizide 5 farklı sayfa vardır; bu yüzden her algoritmanın en az 5 (zorunlu, compulsory) hatası olur. 4 çerçeveyle: FIFO 8, LRU 7, OPT 6 hata — bu dizide anomali yok.

8.2 Slaytlardaki 7. problem: sayfa hatalı EAT ve blok hesabı

(a) $t_m = 100$ ns, $t_{\text{fault}} = 6$ ms $= 6\,000\,000$ ns, $p = 0.0002$:

$$\text{EAT} = 0.9998 \times 100 + 0.0002 \times 6\,000\,000 = 99.98 + 1200 = 1299.98 \text{ ns},$$

yani 5000 erişimde yalnızca bir hatayla RAM'den yaklaşık 13 kat yavaş. Yavaşlamanın %5'in altında kalması için: $100 + p\,(6\,000\,000 - 100) < 105 \Rightarrow p < 5/5\,999\,900 \approx 8.3 \times 10^{-7}$; yani en çok yaklaşık 1,2 milyon erişimde bir hata.

(b) Kayıtlar $R = 120$ B, bloklar $B = 4096$ B, kayıt $k = 750$: bayt konumu $750 \times 120 = 90\,000$; blok $\lfloor 90\,000/4096 \rfloor = 21$; blok içi konum $90\,000 - 21 \times 4096 = 3984$. Son bayt $90\,119$'dur ve $\lfloor 90\,119/4096 \rfloor = 22$: kayıt bir blok sınırını *aşar* (21. bloğun 3984–4095 baytları ve 22. bloğun 0–7 baytları); dolayısıyla iki blok okuması gerekir. 40. kayıt 4800. baytta, 1. blokta 704 konumunda başlar ve tamamen sığar.

Çalışma kümesi

Çalışma kümesi boyutları $4, 6, 3, 5$ olan dört süreç $D = 18$ çerçeveye ihtiyaç duyar. Yalnızca $m = 16$ çerçeve varsa $D > m$ olur: boşa çalışma başlayacaktır ve işletim sistemi bir süreci askıya almalıdır.

9 Sık Yapılan Hatalar ve Sınav Stratejisi

Konu	Puanların genellikle kaybedildiği yer
Zamanlama	Bekleme süresi dönüş eksi patlamadır, başlama zamanı değil; SRTF *kalan* süreyle karşılaştırır; RR'de yeni gelenler kesilen süreçten önce girer; ortalamaları unutmak.
Birimler	8 ms $= 8\,000\,000$ ns; 2 ms $= 2000$ μs; sayfalama sorularında KB $= 1024$ B.
Sayfalama	Ofset hiçbir zaman çevrilmez; geçersiz bir sayfa tuzak üretir ve fiziksel adresi yoktur; $a =$ sınır zaten tuzaktır (katı $<$).
Çerçeve tabloları	FIFO: isabet sırayı değiştirmez; LRU: isabet sayfayı en yeni yapar; OPT: ileriye bak. Hataları iki kez sayın.
Semboller	p, 1. haftada G/Ç bekleme kesri, 7. haftada sayfa hatası oranıdır; α, 5. haftada ortalama ağırlığı, 6. haftada TLB isabet oranıdır.
`fork()`	Var olan her süreç sonraki her `fork()`'u yürütür.
Tanımlar	Eşzamanlılık $\neq$ paralellik; karşılıklı dışlama $\neq$ ilerleme $\neq$ sınırlı bekleme; açlık $\neq$ kilitlenme; sabit bağlantı $\neq$ sembolik bağlantı.

Strateji. Önce bütün kâğıdı okuyun (5 dk). En iyi bildiğinizle başlayın. Her sayısal yanıtta formülü yazın, sayıları birimleriyle yerine koyun, hesaplayın ve sonucu kutu içine alın: aritmetik kaysa bile yöntem kısmi puan kazandırır. Gantt şemalarını ve çerçeve tablolarını zamanlarıyla düzgünce çizin. Takılırsanız ilgili formülü yazın ve geçin. Sonda birimleri ve ortalamaları denetlemek için 5 dakika ayırın.

10 Formül Kâğıdı

Hf.	Büyüklük	Formül
1	Toplu işlemede kullanım	$U = \dfrac{t_{\text{çalışma}}}{t_{\text{çalışma}} + t_{\text{hazırlık}}}$
1	Çoklu programlamada kullanım	$U(n) = 1 - p^n$ (p: G/Ç bekleme kesri)
1	Zaman paylaşımında en kötü bekleme	$T_{\text{bekleme}} \approx n\,q$
2	Yoklamanın maliyeti	$f \cdot t_{\text{check}}$
2	Monolitik ve mikroçekirdek	$T_{\text{mono}} = 2t_s + t_w$; $\quad T_\mu = 4t_s + 2t_c + t_w$
3, 5	Bağlam değişimi ek yükü	$\dfrac{s}{q+s}$
3	Art arda n fork	2^n süreç, $2^n - 1$'i yeni
4	Amdahl yasası	hızlanma$(N) \le \dfrac{1}{S + (1-S)/N}$; $\quad$ sınır $\dfrac{1}{S}$
4	Dönüş, bekleme	$T_{\text{dönüş}} = T_{\text{tamamlanma}} - T_{\text{geliş}}$; $\quad T_{\text{bekleme}} = T_{\text{dönüş}} - T_{\text{patlama}}$
4	Yanıt	$T_{\text{yanıt}} = T_{\text{ilk çalışma}} - T_{\text{geliş}}$
4	Kullanım, üretilen iş	$\dfrac{B}{T}$ (meşgul / toplam); $\quad \dfrac{n}{T}$
5	Üstel ortalama	$\tau_{n+1} = \alpha\, t_n + (1-\alpha)\,\tau_n$
5	Round Robin sınırı	sonraki sıra için bekleme $\le (n-1)\,q$
6	Taban ve sınır	$0 \le a <$ sınır $\Rightarrow a_{\text{fiz}} =$ taban $+ a$, değilse tuzak
6	Takas süresi	boyut / aktarım hızı
6	Yüzde 50 kuralı	belleğin $\dfrac{0.5N}{N + 0.5N} = \dfrac{1}{3}$'ü kaybolabilir
6	Sayfalama	$p = \lfloor a/2^n \rfloor$, $d = a \bmod 2^n$, $a_{\text{fiz}} = f \cdot 2^n + d$
6	Sayfa tablosu boyutu	$2^m/2^n$ girdi $\times$ girdi boyu
6	TLB (ders uzlaşımı)	EAT $= \alpha(t_{\text{TLB}} + t_m) + (1-\alpha)(t_{\text{TLB}} + 2t_m) = t_{\text{TLB}} + (2-\alpha)\,t_m$
6	TLB (sadeleştirilmiş)	EAT $= \alpha\, t_m + (1-\alpha)\,2t_m$
6	Bölütleme	$0 \le d <$ sınır$[s] \Rightarrow$ taban$[s] + d$, değilse tuzak
7	Sayfa hatalı EAT	EAT $= (1-p)\,t_m + p\,t_{\text{fault}}$ (p: hata oranı)
7	Doğrudan erişim, k. kayıt	konum $= kR$; blok $= \lfloor kR/B \rfloor$; blok içi $= kR -$ blok $\cdot B$; $kR + R - 1$. baytı denetle
7	Çalışma kümesi	$D = \sum_i \text{WSS}_i > m \Rightarrow$ boşa çalışma

İkinin kuvvetleri: $2^{10} = 1024$, $2^{12} = 4096$, $2^{16} = 65\,536$, $2^{20} \approx 10^6$. **Onaltılık:** bir onaltılık basamak $= 4$ bit; 4 KB'lık sayfalarda ofset son üç onaltılık basamaktır. **Zaman:** 1 ms $= 10^3$ μs $= 10^6$ ns.

11 Deneme Sınavı

Yönerge. 90 dakika, 100 puan. Kısım 1'deki ders uzlaşımlarını kullanın. Her hesabın her adımını gösterin. Çözümler Kısım 12'tedir; bitirmeden bakmayın.

A Bölümü --- Çoktan seçmeli (10 × 2 = 20 puan)

A1. GM-NAA I/O (1956) sık sık ilk işletim sistemi sayılır, çünkü (a) zaman paylaşımını getirdi (b) işleri art arda çalıştıran bir yerleşik monitörü vardı (c) C ile yazılmıştı (d) grafik arayüzü vardı

A2. İkili mod çalışmasında çekirdek modunu gösteren mod biti değeri (a) 0 (b) 1 (c) 2 (d) 3

A3. Aşağıdakilerden hangisi bir sürecin her iş parçacığına *özeldir*? (a) global değişkenler (b) yığın (heap) (c) yığıt (stack) (d) açık dosyalar

A4. Bekleyen bir sürecin istediği disk okuması tamamlandığında süreç hangi duruma geçer? (a) çalışıyor (b) hazır (c) yeni (d) sonlandı

A5. Konvoy etkisi hangisinin zayıflığıdır? (a) FCFS (b) SRTF (c) Round Robin (d) yaşlandırmalı öncelikli zamanlama

A6. 3 ile başlatılmış sayan bir semafor üç özdeş yazıcıyı koruyor. Aynı anda kaç iş parçacığı bir yazıcı kullanabilir? (a) 1 (b) 2 (c) 3 (d) sınırsız

A7. Sayfalama neyi ortadan kaldırır? (a) iç parçalanmayı (b) dış parçalanmayı (c) ikisini de (d) hiçbirini

A8. Hangi sayfa değiştirme algoritması Belady anomalisi gösterebilir? (a) FIFO (b) LRU (c) OPT (d) hiçbiri

A9. Sabit bağlantı (a) bir yol içeren dosyadır (b) aynı düğüm için başka bir dizin girdisidir (c) dosyanın bir kopyasıdır (d) asıl ad silinince boşta kalan bir bağlantıdır

A10. `getpid()` hangi sistem çağrısı türüne aittir? (a) süreç denetimi (b) dosya yönetimi (c) bilgi bakımı (d) koruma

B Bölümü --- Kısa yanıt (4 × 5 = 20 puan)

B1. Program ile süreç arasındaki farkı açıklayın. Bir süreç denetim bloğunda saklanan dört tür bilgiyi sayın.

B2. Bir bağlam değişimi sırasında ne kaydedilir ve geri yüklenir; buna neden ek yük denir? Geçiş süresi $s = 5$ μs ve zaman dilimi $q = 10$ ms için ek yükü hesaplayın.

B3. Kritik bölge problemine bir çözümün üç gereksinimini ifade edin. İki iş parçacığının yürüttüğü `counter++` işleminin neden bir güncellemeyi kaybettirebileceğini kısaca açıklayın.

B4. İşletim sisteminin bir sayfa hatasını işlemek için yaptığı altı adımı sayın. Hataya yol açan komut neden yeniden başlatılmalıdır?

C Bölümü --- Problemler (60 puan)

C1. **[16 puan]** Tek işlemcili bir sisteme dört süreç geliyor (süreler ms): P1 (varış 0, patlama 8), P2 $(1, 4)$, P3 $(2, 2)$, P4 $(4, 5)$. (a) FCFS, (b) SRTF ve (c) $q = 4$ ile Round Robin için Gantt şemasını çizin; her sürecin

dönüş ve bekleme süresini ve ortalama bekleme süresini hesaplayın. (d) Hangi algoritma en küçük ortalama bekleme süresini verir ve bu neden beklenen bir sonuçtur?

C2. **[12 puan]** Bir sistem 16 bitlik mantıksal adresler ve 1 KB'lık sayfalar kullanıyor. Bir sürecin sayfa tablosu: 0. sayfa $\to 7$. çerçeve, $1 \to 2$, $2 \to 9$, $4 \to 5$; diğer bütün sayfalar geçersiz. (a) Ofset için ve sayfa numarası için kaçar bit kullanılır? Adres uzayında kaç sayfa olabilir? (b) 2100, 4500, 3100 ve 1023 (onluk) mantıksal adreslerini çevirin. (c) $t_{\text{TLB}} = 5$ ns, $t_m = 100$ ns ve isabet oranı 0.9 iken etkin erişim süresini hesaplayın. Hangi isabet oranı EAT $= 110$ ns verir?

C3. **[12 puan]** $4, 1, 2, 4, 3, 5, 1, 3, 2, 4, 3, 5$ başvuru dizisi ve başlangıçta boş 3 çerçeve için FIFO, LRU ve OPT çerçeve tablolarını çizin ve sayfa hatalarını sayın.

C4. **[10 puan]** Bellekte şu delikler var (boyut@başlangıç, KB, adres sırasında): $250@200$, $500@600$, $150@1300$, $400@1600$, $300@2200$. A $= 210$, B $= 380$, C $= 290$, D $= 450$ KB süreçleri bu sırayla geliyor. Her birinin ilk uyan, en iyi uyan ve en kötü uyan ile nereye yerleştirildiğini gösterin. Hangi istekler karşılanamaz ve neden?

C5. **[10 puan]** (a) $t_m = 150$ ns, $t_{\text{fault}} = 5$ ms ve sayfa hatası oranı $p = 0.0004$. Etkin erişim süresini hesaplayın. Yavaşlamayı %20'nin altında tutan en büyük p nedir? (b) Bir programın seri kesri $S = 0.3$. 4 ve 10 çekirdekteki en büyük hızlanmasını ve sonsuz sayıda çekirdek için sınırı hesaplayın.

12 Deneme Sınavı: Çözümler

A Bölümü

A1 (b) A2 (a) A3 (c) A4 (b) A5 (a) A6 (c) A7 (b) A8 (a) A9 (b) A10 (c).

Kısa gerekçeler: A3 her iş parçacığının kendi fonksiyon çağrıları zinciri vardır; A4 bekliyor durumundan çalışıyor durumuna ok yoktur; A7 herhangi bir boş çerçeve herhangi bir sayfayı tutabilir, ama son sayfa genellikle dolu değildir; A8 LRU ve OPT yığıt algoritmalarıdır.

B Bölümü

B1. Program diskte duran, komutlardan oluşan edilgen bir dosyadır; süreç ise yürütülmekte olan bir programdır: program sayacı, yazmaçları, kendi belleği (metin, veri, yığın, yığıt) ve kaynakları olan etkin bir varlık. Tek bir program birçok sürecin temeli olabilir. PCB içeriği (herhangi dördü): süreç durumu; PID ve ebeveyn PID; program sayacı; işlemci yazmaçları; zamanlama bilgisi (öncelik, kuyruk işaretçileri); bellek yönetimi bilgisi (taban/sınır ya da sayfa tablosu); hesap bilgisi; G/Ç durumu (açık dosyalar).

B2. Çalışan sürecin bağlamı (program sayacı, yazmaçlar, yığıt işaretçisi, durum, bellek yönetimi bilgisi) PCB'sine kaydedilir ve sıradaki sürecin bağlamı onun PCB'sinden yüklenir. Geçiş sırasında hiçbir süreç yararlı iş yapmaz; bu yüzden saf ek yüktür. $q = 10$ ms $= 10\,000$ μs ile ek yük $s/(q+s) = 5/10\,005 \approx 0.05\%$'tir.

B3. Karşılıklı dışlama (kritik bölgesinde en çok bir iş parçacığı), ilerleme (içeride kimse yoksa yalnızca girmek isteyen iş parçacıkları karar verir ve bu karar sonsuza kadar ertelenmez), sınırlı bekleme (bekleyen bir iş parçacığından önce başkalarının kaç kez girebileceğine bir sınır: açlık yok). `counter++` üç komuttur: özel bir yazmaca oku, ekle, yaz. İki iş parçacığı da herhangi biri yazmadan önce aynı eski değeri okursa ikinci yazma birincinin üzerine yazar ve bir güncelleme kaybolur.

B4. (1) süreç, girdisi geçersiz olan bir sayfaya başvurur; (2) MMU işletim sistemine tuzak üretir, işletim sistemi adresin geçerli olduğunu denetler; (3) işletim sistemi sayfayı yedek depoda bulur ve boş bir çerçeve bulur (ya da bir sayfayı değiştirir); (4) sayfayı diskten çerçeveye okur (bu arada başka süreçler çalışır); (5) sayfa tablosunu günceller (çerçeve numarası, bit v); (6) komutu yeniden başlatır. Hata oluştuğunda komut tamamlanmamıştı; bu yüzden, bu kez sayfa bellekteyken, baştan yeniden yürütülmelidir.

C Bölümü

C1. *(Üretilmiş tablolar; uzlaşımlar: geçiş süresi 0, eşitlikte varışa göre.)*

(a) FCFS



Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	8	8	$8-0=8$	$8-8=0$	0
P2	1	4	12	$12-1=11$	$11-4=7$	7
P3	2	2	14	$14-2=12$	$12-2=10$	10
P4	4	5	19	$19-4=15$	$15-5=10$	10
Ortalama				$46/4=11.5$	$27/4=6.75$	$27/4=6.75$

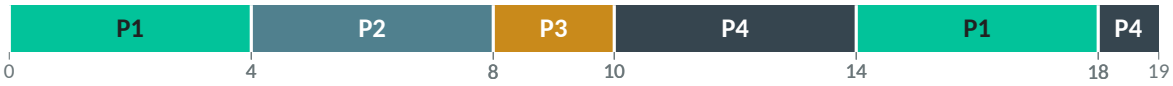
(b) SRTF



$t = 1$: P2'nin ihtiyacı $4 <$ P1'in kalanı 7: kesme. $t = 2$: P3'ün ihtiyacı $2 <$ P2'nin kalanı 3: kesme. $t = 4$: P3 biter; P4 5 ihtiyacıyla gelir; kalanlar P1 7, P2 3: P2 7'ye kadar çalışır, sonra P4 ($5 < 7$) 12'ye kadar, sonra P1 19'a kadar.

Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	8	19	$19 - 0 = 19$	$19 - 8 = 11$	0
P2	1	4	7	$7 - 1 = 6$	$6 - 4 = 2$	0
P3	2	2	4	$4 - 2 = 2$	$2 - 2 = 0$	0
P4	4	5	12	$12 - 4 = 8$	$8 - 5 = 3$	3
Ortalama				$35/4 = 8.75$	$16/4 = 4$	$3/4 = 0.75$

(c) Round Robin, $q = 4$



$t = 4$: kuyruk P2, P3, sonra P4 (tam P1'in zaman dilimi biterken gelir), sonra P1. P2 4–8 (biter), P3 8–10 (biter), P4 10–14 (1 kaldı), P1 14–18 (biter), P4 18–19.

Süreç	Varış	Patlama	Bitiş	Dönüş	Bekleme	Yanıt
P1	0	8	18	$18 - 0 = 18$	$18 - 8 = 10$	0
P2	1	4	8	$8 - 1 = 7$	$7 - 4 = 3$	3
P3	2	2	10	$10 - 2 = 8$	$8 - 2 = 6$	6
P4	4	5	19	$19 - 4 = 15$	$15 - 5 = 10$	6
Ortalama				$48/4 = 12$	$29/4 = 7.25$	$15/4 = 3.75$

(d) SRTF: ortalama bekleme süresi 4 ms; FCFS'de 6.75, RR'de 7.25 ms. Bağlam değişimleri bedava olduğunda SRTF her zaman en küçük ortalama bekleme süresini verir, çünkü kısa bir işi önce çalıştırmak toplam bekleme süresini hiçbir zaman artırmaz (yer değiştirme argümanı). Bedeli: patlama uzunluklarını önceden bilmeyi gerektirir ve uzun iş P1 11 ms bekler.

Puanlama: her algoritma 5 puan (Gantt şeması 2, süreç başına süreler 2, ortalama 1); (d) 1 puan.

C2. (a) 1 KB $= 2^{10}$ B, dolayısıyla ofset $n = 10$ bit, sayfa numarası $16 - 10 = 6$ bittir: $2^6 = 64$ sayfa. (b) $p = \lfloor a/1024 \rfloor$, $d = a - 1024\,p$:

- 2100: $p = 2$, $d = 2100 - 2048 = 52$; 9. çerçeve: $9 \times 1024 + 52 = 9268$.
- 4500: $p = 4$, $d = 4500 - 4096 = 404$; 5. çerçeve: $5 \times 1024 + 404 = 5524$.
- 3100: $p = 3$ ($d = 28$): 3. sayfa geçersiz $\Rightarrow$ **tuzak**.
- 1023: $p = 0$, $d = 1023$; 7. çerçeve: $7 \times 1024 + 1023 = 8191$ (7. çerçevenin son baytı).

(c) EAT $= 0.9 \times 105 + 0.1 \times 205 = 94.5 + 20.5 = 115$ ns $= 5 + (2 - 0.9) \times 100$. 110 ns için: $5 + (2 - \alpha)\,100 = 110 \Rightarrow 2 - \alpha = 1.05 \Rightarrow \alpha = 0.95$. *Puanlama:* (a) 3, (b) adres başına 1.5, (c) 3.

C3. *(Üretilmiş tablolar.)*

FIFO: 10 hata

başvuru	4	1	2	4	3	5	1	3	2	4	3	5
çerçeve 0	**4**	4	4	4	**3**	3	3	3	**2**	2	2	**5**
çerçeve 1		**1**	1	1	1	**5**	5	5	5	**4**	4	4
çerçeve 2			**2**	2	2	2	**1**	1	1	1	**3**	3
hata?	F	F	F		F	F	F		F	F	F	F

LRU: 9 hata

başvuru	4	1	2	4	3	5	1	3	2	4	3	5
çerçeve 0	**4**	4	4	4	4	4	**1**	1	1	**4**	4	4
çerçeve 1		**1**	1	1	**3**	3	3	3	3	3	3	3
çerçeve 2			**2**	2	2	**5**	5	5	**2**	2	2	**5**
hata?	F	F	F		F	F	F		F	F		F

OPT: 7 hata

başvuru	4	1	2	4	3	5	1	3	2	4	3	5
çerçeve 0	**4**	4	4	4	**3**	3	3	3	3	3	3	3
çerçeve 1		**1**	1	1	1	1	1	1	**2**	**4**	4	4
çerçeve 2			**2**	2	2	**5**	5	5	5	5	5	5
hata?	F	F	F		F	F			F	F		

Sağlama: dizide 5 farklı sayfa var, dolayısıyla en az 5 hata; burada OPT $\le$ LRU $\le$ FIFO. *Puanlama:* algoritma başına 4 puan (tablo 3, sayı 1).

C4. *(Üretilmiş tablo.)*

İstek	İlk uyan	En iyi uyan	En kötü uyan
A $= 210$	$250@200 \to$ kalan 40	$250@200 \to$ kalan 40	$500@600 \to$ kalan 290
B $= 380$	$500@600 \to$ kalan 120	$400@1600 \to$ kalan 20	$400@1600 \to$ kalan 20
C $= 290$	$400@1600 \to$ kalan 110	$300@2200 \to$ kalan 10	$300@2200 \to$ kalan 10
D $= 450$	beklemek zorunda	$500@600 \to$ kalan 50	beklemek zorunda

İlk uyan: 720 KB boş olduğu hâlde D (450) beklemek zorundadır — en büyük delik 300 KB'tır. En kötü uyan: D yine beklemek zorundadır (720 KB boş, en büyük delik 290 KB). En iyi uyan dördünü de yerleştirir ve küçük deliklerde 270 KB bırakır. Bu dış parçalanmadır: toplamda yeterince boş bellek var, ama bitişik değil. *Puanlama:* strateji başına 3 puan, açıklama için 1.

C5. (a) $t_{\text{fault}} = 5$ ms $= 5\,000\,000$ ns:

$$\text{EAT} = 0.9996 \times 150 + 0.0004 \times 5\,000\,000 = 149.94 + 2000 = 2149.94 \text{ ns} \approx 14.3\, t_m.$$

Yavaşlama %20'nin altında: EAT < 180 ns, dolayısıyla $150 + p\,(5\,000\,000 - 150) < 180 \Rightarrow p < 30/4\,999\,850 \approx 6.0 \times 10^{-6}$; yani en çok yaklaşık $166\,700$ erişimde bir hata.

(b) $N = 4$: $1/(0.3 + 0.7/4) = 1/0.475 \approx 2.11$; $N = 10$: $1/(0.3 + 0.07) = 1/0.37 \approx 2.70$; sınır $1/0.3 \approx 3.33$. *Puanlama:* (a) 6 (3 EAT, 3 sınır), (b) 4.

13 Sayılar Nasıl Denetlendi?

Bu kılavuzdaki, slaytlardaki ve örnek kâğıttaki her sayı göz kararı değil, hesapla bulundu. `code/` klasöründe şunlar var:

Dosya	Amaç
`osim.py`	tam olarak ders uzlaşımlarıyla küçük benzeticiler: FCFS, SJF, SRTF, öncelikli, RR; FIFO, LRU, OPT; ilk/en iyi/en kötü uyan; 1–7. haftaların formülleri
`review_problems.py`	slaytlardaki çözümlü problemler ve Kısım 2–8'deki örnekler (çıktı: `out_review.txt`)
`practice_exam.py`	Kısım 11'teki deneme sınavı (çıktı: `out_practice.txt`)
`sample_exam.py`	ayrı örnek kâğıt ve cevap anahtarı (çıktı: `out_sample.txt`)
`gen_tables.py`	"üretilmiş" diye işaretlenen Gantt şemalarını ve tabloları yazar (`tex/*.tex`)
`make_tex_tr.py`	bu tabloların Türkçe başlıklı kopyalarını yazar (`tex_tr/*.tex`)

Örneğin 2. problemin SRTF kısmı, `python3 review_problems.py` çıktısında (program çıktısı İngilizcedir: `arr` = varış, `burst` = patlama, `prio` = öncelik, `first` = ilk çalışma, `done` = bitiş, TAT = dönüş, `wait` = bekleme, `resp` = yanıt):

```
--- SRTF ---
Gantt: [0-2 P1] [2-3 P2] [3-5 P3] [5-8 P2] [8-13 P1] [13-18 P4]
      arr burst prio first done  TAT wait resp
P1      0     7    2     0   13   13    6    0
P2      2     4    1     2    8    6    2    0
P3      3     2    4     3    5    2    0    0
P4      5     5    3    13   18   13    8    8
sum waiting 16, sum turnaround 34, sum response 8
avg waiting 4  avg turnaround 8.5  avg response 2  context switches 5  CPU busy 18/18
     ms
```

Kendi alıştırma problemlerinizi üretmek için betiklerdeki iş kümelerini, başvuru dizilerini ya da delikleri değiştirin ve el hesaplarınızı onlarla denetleyin.

14 İleriye Bakış: 9. Hafta

Vizeden sonra ders uygulamalı hâle geliyor. 9. haftadan 14. haftaya kadar her hafta Linux kabuğunda (shell) çalışacağız ve bu yarının fikirlerinin çoğu komut olarak geri dönecek.

- **9. hafta: Linux Kabuğuna Giriş ve Dosya Sisteminde Gezinme.** Linux edinmek (laboratuvar bilgisayarları, `wsl --install` ile WSL, bir Ubuntu sanal makinesi); terminal, kabuk ve çekirdek (kabuk bir sistem programıdır, 2. hafta); ilk komutlar; `pwd`, `ls`, `cd`; 7. haftanın dizin ağacında mutlak ve göreli yollar; yardım almak.
- 10. hafta: dosya işlemleri ve bağlantılar (yine düğümler); 11. hafta: izinler; 12. hafta: yönlendirme ve borular; 13. hafta: süreçler ve iş denetimi (`ps`, sinyaller; 3. ve 5. haftalar); 14. hafta: kabuk betikleri; 15. hafta: final sınavı.

9. haftadan önce bir laboratuvar bilgisayarında, WSL'de ya da bir sanal makinede Linux terminali açabildiğinizden emin olun.

İleri okuma (isteğe bağlı)

- A. Silberschatz, P. B. Galvin ve G. Gagne, *Operating System Concepts*, 10. baskı, Wiley: Bölüm 1–2 (giriş, yapılar), 3 (süreçler), 4 (iş parçacıkları), 5 (işlemci zamanlama), 6 (eşzamanlama araçları), 9 (ana bellek), 10 (sanal bellek), 13 (dosya sistemi arayüzü).
- R. H. ve A. C. Arpaci-Dusseau, *Operating Systems: Three Easy Pieces*, bölüm PDF'leri:
  - Süreçler: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-intro.pdf`
  - Zamanlama: `https://pages.cs.wisc.edu/~remzi/OSTEP/cpu-sched.pdf`
  - Eşzamanlılık ve iş parçacıkları: `https://pages.cs.wisc.edu/~remzi/OSTEP/threads-intro.pdf`
  - Sayfalama: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-paging.pdf`
  - TLB'ler: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-tlbs.pdf`
  - Değiştirme politikaları: `https://pages.cs.wisc.edu/~remzi/OSTEP/vm-beyondphys-policy.pdf`
  - Dosyalar ve dizinler: `https://pages.cs.wisc.edu/~remzi/OSTEP/file-intro.pdf`